

Pan-European University, Faculty of Informatics

Interim Research Report

Definition of inter-process communication

of the project

**Requirements and formal definition of a low-code
language based on object-centric processes -
LowcodeOCP**

Authors:

Gabriel Juhás, Milan Mladoniczky, Juraj Mažári and Tomáš Kováčik

Contact:

gabriel.juhas@paneurouni.com, milan.mladoniczky@paneurouni.com

Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia
under the project No.

09I03-03-V04-00493

August 31, 2026

Version: v0.2

Abstract

This paper presents the formal semantics for the Petriflow language extended to support Multi-Class Multi-Case Petriflow Systems (MCMC-PS). Unlike monolithic high-level Petri net formalisms, Petriflow utilizes strict encapsulation, where process logic, typed data variables, dynamic allowed value domains, and user interface forms are encapsulated within process blueprints called Petriflow classes. In this work, we extend single-net formalisms to a full multi-class and multi-case system environment. We formally define global system state space, globally addressable tasks and events, dynamic domain constraint management, cross-case sub-form compositions, and inter-case event cascading. Furthermore, we formalize lifecycle operations for deleting cases and classes while evaluating reference integrity through dynamic fail-fast transactional semantics.

Contents

Contents	i
List of Figures	ii
1 Introduction	1
2 Multi-Class Multi-Case Petriflow System Definitions	3
3 Events in a Multi-Class Multi-Case Petriflow System	9
3.1 Case-level Events	16
3.2 Task-level Events	18
3.3 Data-level Events	22
3.4 Interactions between Cases via Transactions of Action Execution and Event Cascades	25
4 Inter-Process Orchestration & Batch Task Operations	29
4.1 Batch Task Reference & Interface Aggregation	30
4.2 Query Mechanics and Abstract Addressability	31
5 Discussion: Architectural Foundations & Comparative Analysis	33
5.1 Composition vs. Autonomous Interaction	33
5.2 Communication vs. Transactional Interaction	33
5.3 Interleaved Event Cascading and Synchronous Step Simulation	34
5.4 Interval Transition Semantics and Call-by-Name / Call-by-Value Data Forms	34
5.5 Expressiveness Trade-offs	35
6 Conclusion	36
A Supplementary Material	40

List of Figures

4.1	Student class.	29
4.2	Thesis class.	30
4.3	Committee class	30

Chapter 1

Introduction

In traditional workflow engines and object-oriented architectures, business logic defined via processes and data objects are often strictly decoupled, leading to fragmented system designs. Petriflow addresses this challenge by implementing the **process-driven programming paradigm** [Juh+25], where explicit workflow lifecycles represented as Petri nets are bound directly to typed data objects.

Conceptually, Petriflow leverages an intuitive Object-Oriented Programming (OOP) analogy to model business entities [Juh+25]:

- **Petriflow Class (Blueprint):** Serves as a static template defining the data structure, the underlying Petri net workflow logic and task forms [Juh+25].
- **Case (Instance, Object):** Represents an active runtime object instance [Juh+25]. Lifecycle operations act similarly to object constructors and destructors, initializing or purging the case state.
- **Task & Data Lifecycle:** Unlike classical Place/Transition nets where transition firing is an instantaneous atomic event, a task in Petriflow represents an activity spanning a time interval [Juh+25]. Execution progresses through lifecycle events (**assign**, **cancel**, **finish**) while dynamically binding, locking, and controlling the accessibility of associated data attributes (e.g., toggling fields between *editable*, *required*, or *visible*) [Juh+25].

This paradigm stands in sharp contrast to established high-level Petri net formalisms, including Colored Petri Nets (CPN) [JK09; GA05], Object-Centric Petri Nets (OCPN) [AB20], and Reference Nets [KWM04; Cab+09]. Although these approaches offer immense modeling power for complex runtime dynamics—such as representing dynamic entities via colored tokens or applying the “nets-as-tokens” paradigm—they operate without a native class-based object abstraction. Instead of encapsulating data, dynamic user interfaces, and behavioral lifecycles within self-contained blueprint classes, execution is managed within a globalized net scope. In practice, this globalized perspective often

degenerates into monolithic "spaghetti" architectures, where active process instances, data references, or nested net tokens become deeply entangled within a single overarching model topology. Consequently, modifying, extending, or replacing specific process behaviors requires altering the centralized runtime structure itself, rather than simply swapping an independent class definition.

In contrast, Petriflow shifts the focus from a runtime "map of the world" to a descriptive "catalogue of blueprints" [Juh+25]. Inspired by object-oriented class systems, a **Petriflow class** is defined not as the entire ecosystem itself, but as the explicit behavioral DNA of a specific species, encapsulating the discrete behavior, data schema, forms, and lifecycle of a specific process-centric object [Juh+25]. Much like how individuals defined by different species definitions dynamically interact with one another across species boundaries, in this paper, we extend this paradigm to demonstrate how dynamic interactions unfold between distinct process-centric objects. By strictly separating the *blueprint* (the Petriflow class) from its *realization* (the active case or object instance), we ensure high encapsulation, modularity, and operational independence [Juh+25].

Thus, while our previous work [Juh+25] established the formal semantics for a single isolated Process-Centric Object (PCO) belonging to a single Petriflow class, real-world enterprise applications consist of complex networks of interacting process instances across diverse object definitions.

Building upon single-net lifecycle semantics, this paper extends Petriflow to **Multi-Class Multi-Case Petriflow Systems (MCMC-PS)**. Specifically, we generalize the single-blueprint formal semantics to handle:

1. Dynamic orchestration across multiple Petriflow classes, enabling case instances and their tasks to be accessed by reference and their events to be triggered,
2. Inter-case transactional event execution, enabling cascading event chains between distinct object instances (belonging to the same or different classes) while guaranteeing systemic transactional integrity,
3. Cross-case sub-form composition, providing single-source-of-truth UI interface sharing across independent process instances.

The remainder of this paper is structured as follows: Section 2 provides the formal mathematical definitions of Multi-Class Multi-Case Petriflow Systems (MCMC-PS). Section 3 provides the formal mathematical definitions of events in a MCMC-PS. Section 4 briefly outlines the application of MCMC-PS through an illustrative example. Section 5 discusses the architectural implications and modeling advantages of this approach. Finally, Section 6 concludes the paper and outlines future research directions.

Chapter 2

Multi-Class Multi-Case Petriflow System Definitions

We use $\mathbb{N}$ to denote the nonnegative integers and $\mathbb{N}^+$ to denote the positive integers. Given two arbitrary sets A and B , B^A denotes the set of all functions from A to B . Given a function f from A to B and $C \subseteq A$, $f|_C$ denotes the restriction of f to C . The symbol 2^A denotes the power set of A . Given a binary relation $R \subseteq A \times A$, R^+ denotes its transitive closure and R^* its reflexive and transitive closure. For any set A , A^* denotes the set of all finite sequences of elements from A including the empty sequence.

Building upon the single-class lifecycle semantics established in [Juh+25], this section formalizes the Multi-Class Multi-Case Petriflow System (MCMC-PS) paradigm. While a single Petriflow class dictates the structural and behavioral template of an object species, an MCMC-PS manages a collection of such class definitions alongside a dynamically evolving population of active case instances and their inter-case interactions. Below, we formalize the mathematical building blocks governing this multi-class, multi-case execution environment.

Before defining the core architecture of a MCMC-PS, we establish a foundation for variable typing by defining the Data Type System as follows:

Definition 1 (Data Type System) *A Data Type System is a set of type names $Types = Types_{stat} \cup Types_{dyn}$, partitioned into static types $Types_{stat}$ (e.g., *String*, *Integer*, *Boolean*) and distinguished dynamic reference type names $Types_{dyn} = \{userList, taskRef, caseRef\}$.*

Throughout the remainder of this paper, we assume a fixed Data Type System $Types$. Further we define Petriflow Classes, with workflow parts inspired by numerous Petri net extensions, such as [JLM07; Vog02; Aal+09; Val78].

Definition 2 (Petriflow Class / Blueprint) *A Petriflow class is a 4-tuple $PCC = (Data, Workflow, Interface, cs_0)$, where:*

1. $Data = (CaseVars, ClassVars, Typing)$, such that:

- (a) *CaseVars* and *ClassVars* are finite disjoint sets of instance-scoped (case) and class-scoped (class) data variables ($\text{CaseVars} \cap \text{ClassVars} = \emptyset$). We denote all variables as $\text{Variables} = \text{CaseVars} \cup \text{ClassVars}$.
- (b) *Typing* : $\text{Variables} \rightarrow \text{Types}$ associates a data type to each variable.
- 2. *Workflow* = (P, T, F, C_+, C_-, W) , where P is a finite set of places, T is a finite set of tasks ($P \cap T = \emptyset$), $F \subseteq (P \times T) \cup (T \times P)$ is a flow relation, $C_+, C_- \subseteq P \times T$ are positive and negative context relations, and $W : F \cup C_+ \cup C_- \rightarrow \mathbb{N}^+ \cup P \cup \text{Variables}$ is a weight function.
- 3. *Interface* : $T \rightarrow 2^{\text{Variables}}$ attaches a set of variables to each task $t \in T$, creating the task form.
- 4. cs_0 is the initial state.

Remark 3 (Class Component Notation) For any given Petriflow class $PCC \in \text{Classes}$, we systematically use the subscript notation P_{PCC} , T_{PCC} , CaseVars_{PCC} , ClassVars_{PCC} (where $\text{Variables}_{PCC} = \text{CaseVars}_{PCC} \cup \text{ClassVars}_{PCC}$), F_{PCC} , Typing_{PCC} , and Interface_{PCC} to denote the respective structural components of that specific class PCC .

Definition 4 (Multi-Class Multi-Case System Architecture) Let IDs be a countable set of instance identifiers. A Multi-Class Multi-Case Petriflow System is a pair $\text{sys} = (\text{Classes}, \text{Cases})$, where:

- 1. *Classes* is a finite set of registered Petriflow class blueprints PCC .
- 2. $\text{Cases} \subseteq \text{Classes} \times IDs$ is a finite set of active case instances (PCC, id) , where $id \in IDs$ is an instance identifier unique within PCC .

Definition 5 (Global System State Space) Given a system $\text{sys} = (\text{Classes}, \text{Cases})$, the global runtime system state S_{sys} is a pair:

$$S_{\text{sys}} = (\mathcal{C}, \mathbf{S}) \quad (2.1)$$

where:

- 1. $\mathcal{C} : \text{Classes} \rightarrow \text{ClassStates}$ is a total function assigning a class-level state c to each registered class $PCC \in \text{Classes}$.
- 2. $\mathbf{S} : \text{Cases} \rightarrow \text{InstanceStates}$ is a total function assigning a local instance state s to each active case $(PCC, id) \in \text{Cases}$.

Definition 6 (Type System Semantic Interpretation) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and a set of system users $Users$, the semantic interpretation function $Val_{S_{sys}}$ assigns a set of allowed values to each type name $type \in Types$. To represent unassigned or undefined variable values, we introduce a distinguished element $null$, which is disjoint from all concrete value domains ($null \notin Users \cup Cases \cup T_{sys}$). In particular:

1. **Static Data Types** ($type \in Types_{stat}$):

$$Val_{S_{sys}}(type) = Dom(type) \cup \{null\}$$

where $Dom(type)$ represents the base value domain defined by the execution environment.

2. **User Lists** ($userList \in Types_{dyn}$):

$$Val_{S_{sys}}(userList) = 2^{Users} \cup \{null\}$$

$Users$ are disjoint from all other value domains. For each class $PCC \in Classes$, the set of roles $Roles_{PCC} \subseteq Variables_{PCC}$ is defined as:

$$Roles_{PCC} = \{v \in Variables_{PCC} \mid Typing_{PCC}(v) = userList\}$$

3. **Case References** ($caseRef \in Types_{dyn}$):

$$Val_{S_{sys}}(caseRef) = 2^{Cases} \cup \{null\}$$

4. **Task References** ($taskRef \in Types_{dyn}$): Let T_{sys} be the set of active tasks across all case instances in $Cases$:

$$T_{sys} = \{((PCC, id), t) \mid (PCC, id) \in Cases \wedge t \in T_{PCC}\}$$

The domain type $taskRef$ evaluates to:

$$Val_{S_{sys}}(taskRef) = 2^{T_{sys}} \cup \{null\}$$

Definition 7 (CaseTask) Given a system architecture $sys = (Classes, Cases)$ and a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a CaseTask τ is an element

$$\tau = ((PCC, id), t) \in T_{sys} \quad (2.2)$$

representing the unique binding of a task $t \in T_{PCC}$ to an active case instance $(PCC, id) \in$

Cases.

Definition 8 (State of an Instance in MCMC-PS) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and an active case instance $(PCC, id) \in Cases$, the local instance state $s = \mathbf{S}(PCC, id)$ is a function defined on $P_{PCC} \cup CaseVars_{PCC} \cup T_{PCC}$ such that:

1. **Control Marking** ($s|_{P_{PCC}}$): The restriction $s|_{P_{PCC}} : P_{PCC} \rightarrow \mathbb{N}$ defines the current token marking of control places in P_{PCC} .
2. **Case Variable Valuation** ($s|_{CaseVars_{PCC}}$): The restriction $s|_{CaseVars_{PCC}}$ assigns to each case variable $x \in CaseVars_{PCC}$ its current value $s(x) \in Val_{S_{sys}}(Typing_{PCC}(x))$.
3. **Task Local States** ($s|_{T_{PCC}}$): The restriction $s|_{T_{PCC}}$ maps each task $t \in T_{PCC}$ to its local runtime state tuple:

$$s(t) = (s_{assigned}(t), s_{editable}(t), s_{required}(t), s_{allowed}(t), s_{consumed}(t)) \quad (2.3)$$

where:

- (a) $s_{assigned}(t) \in Users \cup \{null\}$,
- (b) $s_{editable}(t) : Interface_{PCC}(t) \rightarrow \{0, 1\}$,
- (c) $s_{required}(t) : Interface_{PCC}(t) \rightarrow \{0, 1\}$,
- (d) $s_{allowed}(t)$ maps each variable $x \in Interface_{PCC}(t)$ to a subset of its legal domain, i.e., $s_{allowed}(t)(x) \subseteq Val_{S_{sys}}(Typing_{PCC}(x))$,
- (e) $s_{consumed}(t) : \{p \in P_{PCC} \mid (p, t) \in F_{PCC}\} \rightarrow \mathbb{N}$ maps each input place of task t to the number of currently accumulated/consumed tokens.

Definition 9 (Class State in MCMC-PS) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and a registered class $PCC \in Classes$, the class-level state $c = \mathcal{C}(PCC)$ is a valuation function defined on $ClassVars_{PCC}$ that assigns to each class variable $x \in ClassVars_{PCC}$ its current value $c(x) \in Val_{S_{sys}}(Typing_{PCC}(x))$.

Definition 10 (Cross-Case Extended Interface) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and a CaseTask $\tau_1 = ((PCC_1, id_1), t_1) \in T_{sys}$ with $(PCC_1, id_1) \in Cases$, local instance case state $s_1 = \mathbf{S}(PCC_1, id_1)$, and class state $c_1 = \mathcal{C}(PCC_1)$, its extended cross-case interface $Interface^*(\tau_1, S_{sys})$ and consolidated attributes are defined as follows:

1. **Pure Interface:** For any CaseTask $\tau = ((PCC, id), t) \in T_{sys}$:

$$PureInterface(\tau) = \{((PCC, id), v) \mid v \in Interface_{PCC}(t) \wedge Typing_{PCC}(v) \neq taskRef\}. \quad (2.4)$$

2. **Reference Interface:** For any *CaseTask* $\tau = ((PCC, id), t) \in T_{sys}$:

$$RefInterface(\tau) = \{v \in Interface_{PCC}(t) \mid Typing_{PCC}(v) = taskRef\}. \quad (2.5)$$

3. **Direct Task Reference Relation (*hasRef*):**

The binary relation $hasRef(S_{sys}) \subseteq T_{sys} \times T_{sys}$ is defined as:

$$\tau_1 hasRef(S_{sys}) \tau_2 \iff \exists v \in RefInterface(\tau_1) : \tau_2 \in (s_1 \cup c_1)(v). \quad (2.6)$$

4. **Read-Only Constraint along Reference:**

The predicate $readonly : hasRef(S_{sys}) \rightarrow \{0, 1\}$ is defined as:

$$readonly(\tau_1, \tau_2) = 1 \iff \exists v \in RefInterface(\tau_1) : \tau_2 \in (s_1 \cup c_1)(v) \wedge s_{1,editable}(t_1)(v) = 0. \quad (2.7)$$

5. **Transitive Reachability of Reference Tasks:** Let $hasRef^*(S_{sys})$ be the reflexive and transitive closure of $hasRef(S_{sys})$. The set of reachable reference tasks from τ_1 is:

$$RefTasks^*(\tau_1, S_{sys}) = \{\tau \in T_{sys} \mid \tau_1 hasRef^*(S_{sys}) \tau\}. \quad (2.8)$$

6. **Aggregated Extended Interface:** The aggregated cross-case interface for task τ_1 is:

$$Interface^*(\tau_1, S_{sys}) = \bigcup_{\tau \in RefTasks^*(\tau_1, S_{sys})} PureInterface(\tau). \quad (2.9)$$

7. **Consolidated Attribute Valuations:**

For any tuple $((PCC_k, id_k), v) \in Interface^*(\tau_1, S_{sys})$:

(a) **Effective Required Status ($s_{required}^*(v)$):**

$$s_{required}^*(v) = \begin{cases} 1 & \text{if } \exists \tau' = ((PCC', id'), t') \in RefTasks^*(\tau_1, S_{sys}) : \\ & ((PCC_k, id_k), v) \in PureInterface(\tau') \wedge \\ & \sigma'_{required}(t')(v) = 1, \\ 0 & \text{otherwise.} \end{cases} \quad (2.10)$$

(b) **Effective Editable Status** ($s_{\text{editable}}^*(v)$):

$$s_{\text{editable}}^*(v) = \begin{cases} 0 & \text{if } \exists \tau' = ((PCC', id'), t') \in \text{RefTasks}^*(\tau_1, S_{\text{sys}}) : \\ & ((PCC_k, id_k), v) \in \text{PureInterface}(\tau') \wedge \\ & (\sigma'_{\text{editable}}(t')(v) = 0 \vee \text{readonly}^*(\tau_1, \tau') = 1), \\ 1 & \text{otherwise.} \end{cases} \quad (2.11)$$

(c) **Effective Allowed Values** ($s_{\text{allowed}}^*(v)$):

$$s_{\text{allowed}}^*(v) = \bigcap_{\substack{\tau' = ((PCC', id'), t') \in \text{RefTasks}^*(\tau_1, S_{\text{sys}}) \wedge \\ ((PCC_k, id_k), v) \in \text{PureInterface}(\tau')}} \sigma'_{\text{allowed}}(t')(v) \quad (2.12)$$

where $\sigma' = s' \cup c'$ with $s' = \mathbf{S}(PCC', id')$ and $c' = \mathcal{C}(PCC')$.

Remark 11 (Semantic Resolution of Extended Interface Entries) *Note that elements of $\text{Interface}^*(\tau_1, S_{\text{sys}})$ are structured as pairs $((PCC_k, id_k), v)$. If a variable v belongs to ClassVars , multiple occurrences of the same variable name across different case instances (PCC_k, id_k) may enter the extended interface. They all consistently resolve to the single shared class-level attribute $c(v)$ where $c = \mathcal{C}(PCC_k)$, whereas variables belonging to CaseVars independently resolve to their respective local case states $s_k(v)$ where $s_k = \mathbf{S}(PCC_k, id_k)$.*

Chapter 3

Events in a Multi-Class Multi-Case Petriflow System

So far, we have defined a Process-Centric Class (PCC) as a static structural blueprint containing data declarations ($ClassVars_{PCC}$ and $CaseVars_{PCC}$) and workflow logic. To integrate classes into a running system architecture $sys = (Classes, Cases)$ and dynamically instantiate cases, each class blueprint encapsulates an initial blueprint state cs_0 .

This initial blueprint state serves a dual purpose:

1. It defines the starting valuation for class-level variables (c_0), which are initialized once upon class registration and shared across all instances without being copied into individual cases.
2. It provides an initial state template (s_0) that is copied to form the local state $S(PCC, id)$ whenever a new case instance is created.

Having formally established how registered class states $\mathcal{C}(PCC)$ and active instance states $\mathbf{S}(PCC, id)$ are represented, we now utilize these concepts to formalize cs_0 before defining the system event space $\mathcal{E}_{sys}$ that drives state transitions of a MCMC-PS. Because the definition domain of state function for class state and for instance case state of a PCC are disjoint, we can define initial state cs_0 as a union of such state functions.

Definition 12 (Initial Blueprint State cs_0 of a Petriflow Class) *Given a Petriflow class blueprint $PCC = (Data, Workflow, Interface, cs_0)$, its initial blueprint state cs_0 is defined as a pair:*

$$cs_0 = c_0 \cup s_0 \quad (3.1)$$

where c_0 represents the initial class-level data valuation component and s_0 represents the default instance state template.

1. Initial Class-Level Valuation (c_0):

A valuation function c_0 such that for each class variable $x \in ClassVars_{PCC}$, $c_0(x) \in Val_{S_{sys}}(Typing_{PCC}(x))$ assigns its defined initial value (or null if unassigned).

2. **Default Initial Instance State Template** (s_0):

A function defined on $P_{PCC} \cup \text{CaseVars}_{PCC} \cup T_{PCC}$ serving as the initial state template for any newly created case instance of class PCC , such that:

- (a) **Initial Control Marking** ($s_0|_{P_{PCC}}$): $s_0|_{P_{PCC}} = M_0$, where $M_0 : P_{PCC} \rightarrow \mathbb{N}$ is the initial token marking of the workflow.
- (b) **Initial Case Variable Valuation** ($s_0|_{\text{CaseVars}_{PCC}}$): For each case variable $x \in \text{CaseVars}_{PCC}$, $s_0(x) \in \text{Val}_{S_{sys}}(\text{Typing}_{PCC}(x))$ assigns its default valuation specified in the blueprint (or null if unassigned).
- (c) **Initial Task Local States** ($s_0|_{T_{PCC}}$): For each task $t \in T_{PCC}$, $s_0(t) = (s_{assigned}^0(t), s_{editable}^0(t), s_{required}^0(t), s_{allowed}^0(t), s_{consumed}^0(t))$ where:
 - (a) $s_{assigned}^0(t) = \text{null}$,
 - (b) $s_{editable}^0(t)(v) \in \{0, 1\}$ for all $v \in \text{Interface}_{PCC}(t)$ specifies the initial editability flag for tasks defined in PCC ,
 - (c) $s_{required}^0(t)(v) \in \{0, 1\}$ for all $v \in \text{Interface}_{PCC}(t)$ specifies the initial requirement flag for tasks defined in PCC ,
 - (d) $s_{allowed}^0(t)(v) \subseteq \text{Val}_{S_{sys}}(\text{Typing}_{PCC}(v))$ for all $v \in \text{Interface}_{PCC}(t)$ specifies the initial allowed value domain for tasks defined in PCC ,
 - (e) $s_{consumed}^0(t)(p) = 0$ for all $p \in P_{PCC}$ such that $(p, t) \in F_{PCC}$.

Definition 13 (System Event Space $\mathcal{E}_{sys}$) Given a system architecture $sys = (\text{Classes}, \text{Cases})$, the global system event space $\mathcal{E}_{sys}$ driving state transitions in an MCMC-PS is partitioned into four distinct event categories:

$$\mathcal{E}_{sys} = \mathcal{E}_{classes} \cup \mathcal{E}_{cases} \cup \mathcal{E}_{tasks} \cup \mathcal{E}_{data} \quad (3.2)$$

where:

1. **Class-level Events** ($\mathcal{E}_{classes}$): Operations managing the registration and removal of class blueprints:

$$\begin{aligned} \mathcal{E}_{classes} = \{ & \text{registerClass}(PCC) \mid PCC \notin \text{Classes} \} \cup \\ & \{ \text{removeClass}(PCC) \mid PCC \in \text{Classes} \} \end{aligned}$$

2. **Case-level Events** ($\mathcal{E}_{cases}$): Operations governing the lifecycle of case instances:

$$\begin{aligned} \mathcal{E}_{cases} = \{ & \text{createCase}(PCC, id) \mid PCC \in \text{Classes} \wedge (PCC, id) \notin \text{Cases} \} \cup \\ & \{ \text{deleteCase}(PCC, id) \mid (PCC, id) \in \text{Cases} \} \end{aligned}$$

3. **Task-level Events** ($\mathcal{E}_{tasks}$): Operations controlling task interaction and workflow execution:

$$\begin{aligned} \mathcal{E}_{tasks} = \{ & viewTask((PCC, id), t), assignTask((PCC, id), t, u), \\ & finishTask((PCC, id), t), cancelTask((PCC, id), t) \mid \\ & (PCC, id) \in Cases, t \in T_{PCC}, u \in Users \} \end{aligned}$$

4. **Data-level Events** ($\mathcal{E}_{data}$): Operations handling data access, variable mutations, and task-local interface permission constraints within active *CaseTask* contexts:

$$\begin{aligned} \mathcal{E}_{data_val} = \{ & getData((case, t), (case_k, v)), \\ & setData((case, t), (case_k, v), value) \mid \\ & (case, t) = ((PCC, id), t) \in T_{sys}, \\ & case_k = (PCC_k, id_k) \in Cases, v \in Variables_{PCC_k}, \\ & value \in Val_{S_{sys}}(Typing_{PCC_k}(v)) \}, \\ \mathcal{E}_{data_allow} = \{ & setAllowedData((case, t), w, D) \mid \\ & (case, t) = ((PCC, id), t) \in T_{sys}, \\ & w \in Interface_{PCC}(t), D \subseteq Val_{S_{sys}}(Typing_{PCC}(w)) \}, \\ \mathcal{E}_{data} = & \mathcal{E}_{data_val} \cup \mathcal{E}_{data_allow} \end{aligned}$$

The event space $\mathcal{E}_{sys}$ categorizes all admissible operational events driving the state transitions of the system *sys*.

In this formulation, each active case instance $case \in Cases$ is explicitly represented as a pair $case = (PCC, id)$, where $PCC \in Classes$ identifies its parent blueprint and $id \in IDs$ uniquely identifies the instance. This explicit pairing directly links the execution context of task-level ($\mathcal{E}_{tasks}$) and data-level ($\mathcal{E}_{data}$) events to the structural scope of the underlying class blueprint PCC .

Furthermore, data mutation events $setData(\tau, (case_k, v), value) \in \mathcal{E}_{data_val}$ operate over the combined variable space of the targeted case instance:

$$Variables_{PCC_k} = CaseVars_{PCC_k} \cup ClassVars_{PCC_k}$$

where $target_case(e) = case_k = (PCC_k, id_k)$. Consequently, data modifications executed within an assigned task of source case can uniformly target both instance-specific state variables ($CaseVars_{PCC_k}$) and shared class-level variables ($ClassVars_{PCC_k}$) of the targeted case instance.

Similarly, permission mutation events $setAllowedData(\tau, w, D) \in \mathcal{E}_{data_allow}$ also operate

on the target case context, where the target context strictly coincides with the source context ($target_case(e) = source_case(e) = case$). These events modify the local task interface state component $s_{allowed}(t)(w)$ (Definition 7) for a variable $w \in Interface_{PCC}(t)$ displayed in the form interface of the active task t .

Building upon the operational semantics presented in [3], we extend the state transition framework to the multi-class multi-case system architecture (MCMC-PS). For a given system architecture $sys = (Classes, Cases)$, a global system state $S_{sys} = (\mathcal{C}, \mathbf{S})$, an event $e \in \mathcal{E}_{sys}$, and an initiating user $u \in Users$, we formally specify:

1. **Triggerability:** Whether event e is triggerable in state S_{sys} under architecture sys based strictly on structural, state-based, and data readiness, expressed by the predicate $triggerable(sys, S_{sys}, e)$.
2. **Authorization:** Whether user u is authorized to trigger event e in state S_{sys} under architecture sys , expressed by the predicate $authorized(sys, S_{sys}, e, u)$.
3. **State Transition:** The resulting updated architecture sys' and target global state S'_{sys} obtained upon triggering a triggerable event e , denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$.

Definition 14 (Event Context Extractors & Scope Restrictions) *To formally map any system event $e \in \mathcal{E}_{sys}$ to its structural and operational scopes, we define the source and target context extractors:*

1. **Source Context (Initiating Scope / Authorization Context):** *Determines the case instance where the event is initiated and against which user authorization is validated:*

$$source_case(e) = \begin{cases} case & \text{if } e \text{ operates directly on } case \in Cases \\ case & \text{if } e \in \mathcal{E}_{data} \text{ is initiated within } \tau = (case, t) \\ \perp & \text{if } e \in \mathcal{E}_{classes} \end{cases}$$

The source class $source_class(e)$ is derived directly from the referenced blueprint or the initiating case:

$$source_class(e) = \begin{cases} PCC & \text{if } e \in \{registerClass(PCC), \\ & \quad removeClass(PCC)\} \\ PCC & \text{where } source_case(e) = (PCC, id) \text{ otherwise} \end{cases}$$

Target Context (Data Mutation Scope): *For cross-case valuation events $e \in \mathcal{E}_{data_val}$, the target case instance and class blueprint whose state variable v is*

read or updated are given by:

$$\text{target_case}(e) = \text{case}_k = (PCC_k, id_k), \quad \text{target_class}(e) = PCC_k$$

For all local interface events and control transitions $e \in \mathcal{E}_{sys} \setminus \mathcal{E}_{data_val}$ (including *setAllowedData*), the target context strictly coincides with the source context ($\text{target_case}(e) = \text{source_case}(e)$ and $\text{target_class}(e) = \text{source_class}(e)$).

Given any event subset $X \subseteq \mathcal{E}_{sys}$, the **restricted event spaces** relative to a class *PCC* or case *case* are defined as:

$$X|_{PCC} \equiv \{e \in X \mid \text{source_class}(e) = PCC \vee \text{target_class}(e) = PCC\}$$

$$X|_{case} \equiv \{e \in X \setminus \mathcal{E}_{classes} \mid \text{source_case}(e) = case\}$$

For the sake of simplicity, registering a new class blueprint is not bound to any existing case or class-level role (as roles are defined as data variables of type *userList* with values being subsets of users $2^{Users} \cup \{null\}$ within a specific *PCC* blueprint). To govern the execution of *registerClass* events, as well as *removeClass* events, we assume that a dedicated subset of administrative users $Admins \subseteq Users$ is defined within the global system environment.

Definition 15 (Triggering *registerClass*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (C, S)$, a user $u \in Users$, and an event $e = \text{registerClass}(PCC) \in \mathcal{E}_{classes}$ where $PCC = (Data, Workflow, Interface, cs_0)$ with initial blueprint state $cs_0 = (c_0, PCC, s_0, PCC)$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $\text{triggerable}(sys, S_{sys}, e) = true$, if and only if:

$$PCC \notin Classes$$

2. **Authorization:** User u is authorized to trigger event e under architecture sys and state S_{sys} , expressed by $\text{authorized}(sys, S_{sys}, e, u) = true$, if and only if:

$$u \in Admins$$

3. **State Transition:** When e is triggered, it yields an updated system architecture $sys' = (Classes', Cases)$ and global state $S'_{sys} = (C', S)$, denoted as $(sys', S'_{sys}) =$

$trigger(sys, S_{sys}, e)$, such that:

$$\begin{aligned} Classes' &= Classes \cup \{PCC\} \\ \mathcal{C}'(PCC) &= c_{0,PCC} \quad \text{and} \quad \mathcal{C}'(PCC') = \mathcal{C}(PCC') \quad \forall PCC' \in Classes \end{aligned}$$

while the set of active cases $Cases$ and their instance states $\mathbf{S}$ remain unchanged.

The class removing operation allows an authorized user to unregister a class blueprint from the system, provided that no active cases of that class exist.

Definition 16 (Triggering $removeClass$) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a user $u \in Users$, and an event $e = removeClass(PCC) \in \mathcal{E}_{classes}$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if:

$$PCC \in Classes \quad \text{and} \quad \nexists case = (PCC, id) \in Cases$$

2. **Authorization:** User u is authorized to trigger event e under architecture sys and state S_{sys} , expressed by $authorized(sys, S_{sys}, e, u) = true$, if and only if:

$$u \in Admins$$

3. **State Transition:** When e is triggered, it yields an updated system architecture $sys' = (Classes', Cases)$ and global state $S'_{sys} = (\mathcal{C}', \mathbf{S})$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, such that:

$$\begin{aligned} Classes' &= Classes \setminus \{PCC\} \\ \mathcal{C}'(PCC') &= \mathcal{C}(PCC') \quad \forall PCC' \in Classes' \end{aligned}$$

while the set of active cases $Cases$ and their instance states $\mathbf{S}$ remain unchanged.

Due to strict encapsulation, all case-level and task-level events must be triggered exclusively by users authorized within the context of the respective class blueprint $PCC \in Classes$. Recalling that roles are represented as data variables of type $userList$ (where $Val_{S_{sys}}(userList) = 2^{Users} \cup \{\text{null}\}$, with $Roles = \{v \in Variables \mid Typing(v) = userList\}$), access control is uniformly governed by a role reference mapping $RoleRef$.

Definition 17 (Role Reference Mapping) Let $PCC \in Classes$ be a class blueprint. The domain of the role reference mapping for PCC , denoted as $Domain(RoleRef_{PCC})$, is

defined as:

$$\text{Domain}(\text{RoleRef}_{PCC}) = \mathcal{E}_{\text{cases}}|_{PCC} \cup \mathcal{E}_{\text{tasks}}|_{PCC} \cup T_{PCC} \cup \mathcal{E}_{\text{data}}|_{PCC}$$

The role reference mapping RoleRef_{PCC} for a class blueprint PCC is a function:

$$\text{RoleRef}_{PCC} : \text{Domain}(\text{RoleRef}_{PCC}) \rightarrow \text{Roles}_{PCC} \cup \{\text{Users}\}$$

which assigns an authorization constraint - either a role variable $r \in \text{Roles}_{PCC}$ or the entire user set Users - to case-level events, task-level events, individual tasks, and data-level events associated with PCC .

Specifically, for any data event $e \in \mathcal{E}_{\text{data}}|_{PCC}$ associated with a *CaseTask* context $\tau = (\text{case}, t) \in T_{\text{sys}}$, where $\text{case} = (PCC, id) \in \text{Cases}$ and

$$e \in \{\text{getData}(\tau, (\text{case}_k, v)), \text{setData}(\tau, (\text{case}_k, v), \text{value}), \\ \text{setAllowedData}((\text{case}, t), w, D)\},$$

the role reference mapping is strictly derived from the underlying task t :

$$\text{RoleRef}_{PCC}(e) = \text{RoleRef}_{PCC}(t)$$

In addition to event execution, mapping roles directly to individual tasks $t \in T$ serves to control data interaction. Specifically, as defined above $\text{RoleRef}_{PCC}(t)$ specifies the set of users authorized to access and manipulate the data variables declared in the extended interface $\text{Interface}^*(\tau, S_{\text{sys}})$ of an active *CaseTask* $\tau = ((PCC, id), t) \in T_{\text{sys}}$.

Remark 18 (Authorization Expressions and Petriflow Implementation) *In the reference implementation of the MCMC-PS framework within the Netgrif Petriflow engine, authorization constraints can be defined as arbitrary set-based role expressions—combining local case-level roles and global class-level roles via set operations such as unions, intersections, and set differences (e.g., $r_{\text{case}} \setminus r_{\text{blacklisted}}$). For formal simplicity and mathematical clarity in this specification, RoleRef_{PCC} maps each event or task to a single representative role variable $r \in \text{Roles}_{PCC}$ (or Users). This abstraction preserves the structural authorization semantics without loss of generality.*

To simplify notation, when a Petriflow class PCC is clear from the context, we omit the subscript $\cdot_{PCC}$.

Definition 19 (Valuation Function val) *Let $\text{sys} = (\text{Classes}, \text{Cases})$ be a system architecture, $S_{\text{sys}} = (\mathcal{C}, \mathbf{S})$ be the global state, and $\text{case} = (PCC, id) \in \text{Cases}$ be a case instance.*

The valuation function $val(sys, S_{sys}, case, x)$ resolves a given role reference, variable, or place x within global state S_{sys} , defined as:

$$val(sys, S_{sys}, case, x) = \begin{cases} Users & \text{if } x = Users \\ s(x) & \text{if } x \in P \cup CaseVars \quad (\text{where } s = \mathbf{S}(case)) \\ c(x) & \text{if } x \in ClassVars \quad (\text{where } c = \mathcal{C}(PCC)) \end{cases}$$

Where context uniquely determines any subset of the arguments $(sys, S_{sys}, case)$, implicit parameters may be omitted or moved to subscripts for brevity. Accordingly, we write $val(sys, S_{sys}, case, x)$ simply as $val_{case}(x)$, $val_{S_{sys}}(x)$, or $val(x)$ whenever no ambiguity arises.

Definition 20 (User Authorization of Events) Let $sys = (Classes, Cases)$ be a system architecture, $S_{sys} = (\mathcal{C}, \mathbf{S})$ be the global state, $u \in Users$ be a user, and $e \in \mathcal{E}_{sys} \setminus \mathcal{E}_{class}$ be an instance-level event in context $\tau = (case, t) \in T_{sys}$ with $case = (PCC, id)$.

A user u is authorized to trigger event e under architecture sys and state S_{sys} , denoted as $authorized(sys, S_{sys}, u, e) = true$, if and only if all of the following conditions hold:

1. **Role-Based Permission:** User u holds the required role assigned to the event:

$$u \in val(RoleRef(e))$$

2. **Assigned Task Ownership:**

If $e \in \mathcal{E}_{data}|_{case} \cup \{finishTask(\tau), cancelTask(\tau)\}$, user u must be the active assigned owner of task t within the source context $case$:

$$s_{assigned}(t) = u$$

where $s = \mathbf{S}(case)$.

3.1 Case-level Events

Case-level events ($\mathcal{E}_{cases}$) govern the dynamic creation and destruction of workflow instances within registered class blueprints. Unlike class-level operations, creating a new case instance does not alter the available class blueprints $Classes$, but directly mutates the active case domain $Cases$ and extends the instance state function $\mathbf{S}$.

Since user authorization for all events $e \in \mathcal{E}_{cases} \cup \mathcal{E}_{tasks}$ is uniformly governed by Definition 20, the subsequent definitions focus exclusively on their triggerability conditions and state transitions.

Definition 21 (Triggering *createCase*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a user $u \in Users$, and an event $e = createCase(PCC, id) \in \mathcal{E}_{cases}$ where $PCC = (Data, Workflow, Interface, cs_0) \in Classes$ with initial blueprint state $cs_0 = (c_{0,PCC}, s_{0,PCC})$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if:

$$PCC \in Classes \quad \text{and} \quad (PCC, id) \notin Cases$$

2. **State Transition:** When e is triggered, it yields an updated system architecture $sys' = (Classes, Cases')$ and global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, such that:

$$\begin{aligned} Cases' &= Cases \cup \{(PCC, id)\} \\ \mathbf{S}'((PCC, id)) &= s_{0,PCC} \\ \mathbf{S}'(case') &= \mathbf{S}(case') \quad \forall case' \in Cases \end{aligned}$$

while the class valuation function $\mathcal{C}$ remains unchanged.

To ensure referential integrity during case destruction, a case instance cannot be deleted arbitrarily. If other active cases or class variables contain references (*caseRef* or *taskRef*) pointing to the target case or its associated tasks, removing it would yield dangling references in S_{sys} .

We formally define the auxiliary predicate $referenced(sys, S_{sys}, case)$ to verify that no active references exist prior to triggering a case deletion event:

Definition 22 (Referenced Case Predicate) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and a target case instance $case = (PCC, id) \in Cases$, the predicate $referenced(sys, S_{sys}, case) = true$ holds if and only if:

$$\begin{aligned} &\exists PCC' \in Classes, x \in ClassVars_{PCC'} : \\ &\quad (Typing_{PCC'}(x) = caseRef \wedge case \in \mathcal{C}(PCC')(x)) \vee \\ &\quad (Typing_{PCC'}(x) = taskRef \wedge \exists t \in T_{PCC} : (case, t) \in \mathcal{C}(PCC')(x)) \\ &\vee \exists case' = (PCC', id') \in Cases, x \in CaseVars_{PCC'} : \\ &\quad (Typing_{PCC'}(x) = caseRef \wedge case \in \mathbf{S}(case')(x)) \vee \\ &\quad (Typing_{PCC'}(x) = taskRef \wedge \exists t \in T_{PCC} : (case, t) \in \mathbf{S}(case')(x)) \end{aligned}$$

Definition 23 (Triggering *deleteCase*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a user $u \in Users$, and an event $e = deleteCase(case) \in \mathcal{E}_{cases}$ where $case = (PCC, id)$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if:

$$case \in Cases \quad \text{and} \quad \neg referenced(sys, S_{sys}, case)$$

2. **State Transition:** When e is triggered, it yields an updated system architecture $sys' = (Classes, Cases')$ and global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, such that:

$$Cases' = Cases \setminus \{case\}$$

$$\mathbf{S}' = \mathbf{S}|_{Cases'}$$

while the class valuation function $\mathcal{C}$ remains unchanged.

3.2 Task-level Events

Before presenting the operational semantics of task-level events, we recall the definition of enabled tasks from [Juh+25].

Definition 24 (Enabled Task) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and a case task $\tau = (case, t) \in \mathcal{T}_{sys}$ where $case = (PCC, id) \in Cases$ with local instance state $s = \mathbf{S}(case)$. Let the workflow of PCC be $Workflow = (P, T, F, C_+, C_-, W)$ and data be $Data = (CaseVars, ClassVars, Typing)$.

We say that task $t \in T$ is enabled in state S_{sys} , expressed as $enabled(sys, S_{sys}, \tau) = true$, if and only if:

1. **Unassigned status:**

$$s_{assigned}(t) = null$$

2. **Positive flow and context relations ($F \cup C_+$):**

$$\forall (p, t) \in F \cup C_+ \text{ with } W((p, t)) \in \mathbb{N}^+ : \quad s(p) \geq W((p, t))$$

$$\forall (p, t) \in F \cup C_+ \text{ with } x = W((p, t)) \in P \cup Variables :$$

$$s(W((p, t))) \in \mathbb{N} \quad \wedge \quad s(p) \geq val(x)$$

3. **Negative context relations (C_-):**

$$\forall (p, t) \in C_- \text{ with } W((p, t)) \in \mathbb{N}^+ : \quad s(p) < W((p, t))$$

$$\forall (p, t) \in C_- \text{ with } x = W((p, t)) \in P \cup Variables :$$

$$s(W((p, t))) \in \mathbb{N} \quad \wedge \quad s(p) < val(x)$$

Inspecting a task via *viewTask* is a side-effect-free operation that can be executed as long as the task is ready or currently in progress.

Definition 25 (Triggering *viewTask*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and an event $e = viewTask((PCC, id), t) \in \mathcal{E}_{tasks}$ where $case = (PCC, id) \in Cases$, $t \in T_{PCC}$, and $s = \mathbf{S}(case)$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if:

$$t \text{ is enabled in } s \quad \vee \quad s_{case, assigned}(t) \neq null$$

2. **State Transition:** When e is triggered, it yields an unchanged system architecture $sys' = sys$ and global state $S'_{sys} = S_{sys}$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, such that:

$$S'_{sys} = S_{sys}$$

Intuitively, the *assignTask* event assigns an enabled task to a target user and consumes tokens from its input places to lock the required resources. The formal specification is structured as follows:

Definition 26 (Triggering *assignTask*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and an event $e = assignTask((PCC, id), t, x) \in \mathcal{E}_{tasks}$ where $case = (PCC, id) \in Cases$, $t \in T_{PCC}$, $x \in Users$, and current case state $s = \mathbf{S}(case)$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if:

$$t \text{ is enabled in } s \wedge s_{assigned}(t) = null \wedge x \in val_{case}(RoleRef(t))$$

2. **State Transition:** When e is triggered, it yields an unchanged architecture $sys' = sys$ and an updated global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, where $\mathbf{S}'(case') = \mathbf{S}(case')$ for all $case' \neq case$, and the updated case state $s' = \mathbf{S}'(case)$ differs from s as follows:

- $\forall (p, t) \in F$ with $W(p, t) \in \mathbb{N}_0$:

$$s'(p) = s(p) - W(p, t) \quad \text{and} \quad s'_{consumed}(t)(p) = W(p, t)$$

- $\forall(p, t) \in F$ with $W(p, t) \in P \cup \text{Variables}$:

$$s'(p) = s(p) - \text{val}_{S_{sys}}(W(p, t))$$

$$s'_{consumed}(t)(p) = \text{val}_{S_{sys}}(W(p, t))$$

- The assignment state is updated to:

$$s'_{assigned}(t) = x$$

The *cancelTask* event releases a currently assigned task, returning all consumed tokens from its buffer back to their respective input places and clearing the user assignment so that the task becomes available for execution again.

It is important to note that changes induced by data variable assignments during the task's execution are not reverted upon cancellation in the current version of Petriflow. While restoring place markings is straightforward, reverting side effects caused by chained actions (e.g., pre- and post-events, secondary assignments, or external API calls) presents a non-trivial challenge that remains an open problem in workflow research.

The formal operational semantics of *cancelTask* is specified as follows:

Definition 27 (Triggering *cancelTask*) Given a system architecture $sys = (\text{Classes}, \text{Cases})$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, and an event $e = \text{cancelTask}((PCC, id), t) \in \mathcal{E}_{tasks}$ where $case = (PCC, id) \in \text{Cases}$, $t \in T_{PCC}$, and current case state $s = \mathbf{S}(case)$:

1. **Triggerability:** The event e is triggerable under architecture sys and state S_{sys} , expressed by $\text{triggerable}(sys, S_{sys}, e) = \text{true}$, if and only if:

$$s_{assigned}(t) \neq \text{null}$$

2. **State Transition:** When e is triggered, it yields an unchanged architecture $sys' = sys$ and an updated global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = \text{trigger}(sys, S_{sys}, e)$, where $\mathbf{S}'(case') = \mathbf{S}(case')$ for all $case' \neq case$, and the updated case state $s' = \mathbf{S}'(case)$ differs from s as follows:

- Tokens locked in the consumed buffer are restored to their input places:

$$\forall(p, t) \in F : \quad s'(p) = s(p) + s_{consumed}(t)(p)$$

- The consumed buffer for task t is reset to zero for all input places:

$$\forall(p, t) \in F : \quad s'_{consumed}(t)(p) = 0$$

- The assignment state is cleared:

$$s'_{assigned}(t) = null$$

The *finishTask* event concludes the execution of an assigned task. Beyond checking that the task is currently assigned, its triggerability relies on validation guards: all mandatory data fields across the extended interface must be populated, and any dynamic outgoing arc weights must evaluate to valid non-negative integers. Upon successful execution, new tokens are produced in the output places, the locked input resources are released, and the assignment is cleared.

The formal operational semantics of *finishTask* is defined as follows:

Definition 28 (Triggering *finishTask*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a task instance $\tau = (case, t) \in T_{sys}$ where $case = (PCC, id) \in Cases$ and $t \in T_{PCC}$, and current case state $s = \mathbf{S}(case)$:

1. **Triggerability:** The event $e = finishTask(case, t) \in \mathcal{E}_{tasks}$ is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if all of the following conditions hold:

$$s_{assigned}(t) \neq null$$

$$\forall (case_k, v) \in Interface^*(\tau, S_{sys}) :$$

$$(s^*_{required}(v) = 1 \implies val_{case_k}(v) \neq null) \wedge$$

$$(val_{case_k}(v) \neq null \implies val_{case_k}(v) \in s^*_{allowed}(v))$$

$$\forall (t, p) \in F \text{ with } W(t, p) \in P \cup Variables : \quad val_{case}(W(t, p)) \in \mathbb{N}$$

2. **State Transition:** When e is triggered, it yields an unchanged architecture $sys' = sys$ and an updated global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = trigger(sys, S_{sys}, e)$, where $\mathbf{S}'(case') = \mathbf{S}(case')$ for all $case' \neq case$, and the updated case state $s' = \mathbf{S}'(case)$ differs from s as follows:

- $\forall (t, p) \in F \text{ with } W(t, p) \in \mathbb{N}:$

$$s'(p) = s(p) + W(t, p)$$

- $\forall (t, p) \in F \text{ with } W(t, p) \in P \cup Variables:$

$$s'(p) = s(p) + val_{S_{sys}}(W(t, p))$$

- The consumed buffer for task t is cleared for all input places:

$$\forall (p, t) \in F : \quad s'_{consumed}(t)(p) = 0$$

- The assignment state is cleared:

$$s'_{assigned}(t) = null$$

3.3 Data-level Events

Definition 29 formalizes how data variables are updated within the MCMC-PS framework. It ensures that data integrity is maintained by strictly linking data modification to task ownership and explicit interface permissions:

- **Authorization (1):** Authorization is tied directly to the active *CaseTask* context $\tau = (case, t) \in T_{sys}$. Whenever a user u has a task assigned, which is expressed by $s_{assigned}(t) = u$, the user is authorized to manipulate data within that task's context (see Definition 20).
- **Strict Context for Editing:** A *setData* event is triggerable only if the target variable-case pair $(case_k, v)$ belongs to the task's cross-case extended interface $(Interface^*(\tau, S_{sys}))$, is explicitly marked as editable ($s_{editable}^*(v) = 1$), and the new value is valid according to $s_{allowed}^*(v)$.
- **Atomic Data Update (2):** When triggered, the variable is updated atomically, modifying either the local case state $s'_k(v)$ for *CaseVars* or the shared class-level state $c'_k(v)$ for *ClassVars*.

Definition 29 (Triggering *setData*) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathcal{S})$, a *CaseTask* instance $\tau = (case, t) \in T_{sys}$, a target variable-case pair $(case_k, v) \in Interface^*(\tau, S_{sys})$, and a value $\in Val_{S_{sys}}(Typing_{PCC_k}(v))$:

1. Triggerability:

The event $e = setData((case, t), (case_k, v), value) \in \mathcal{E}_{data}$ is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if all of the following conditions hold:

$$s_{assigned}(t) \neq 0 \quad (Assigned\ Task)$$

$$(case_k, v) \in Interface^*(\tau, S_{sys}) \quad (Interface\ Scope)$$

$$s_{editable}^*(v) = 1 \quad (Editability\ Permission)$$

$$value \neq val_{case_k}(v) \quad (State\ Change\ Necessity)$$

$$value \neq null \implies value \in s_{allowed}^*(v) \quad (Allowed\ Values\ Constraint)$$

where $s = \mathbf{S}(\text{case})$.

2. **State Transition:** When e is triggered, it yields an unchanged architecture $\text{sys}' = \text{sys}$ and an updated global state $S'_{\text{sys}} = (\mathcal{C}', \mathbf{S}')$, denoted as $(\text{sys}', S'_{\text{sys}}) = \text{trigger}(\text{sys}, S_{\text{sys}}, e)$, as follows:

- If $v \in \text{CaseVars}_{PCC_k}$, then $\mathcal{C}' = \mathcal{C}$, and $\mathbf{S}'$ differs from $\mathbf{S}$ only at $\text{case}_k = (PCC_k, id_k)$, where the updated case state $s'_k = \mathbf{S}'(\text{case}_k)$ satisfies:

$$s'_k(v) = \text{value}$$

- If $v \in \text{ClassVars}_{PCC_k}$, then $\mathbf{S}' = \mathbf{S}$, and $\mathcal{C}'$ differs from $\mathcal{C}$ only at PCC_k , where the updated class state $c'_k = \mathcal{C}'(PCC_k)$ satisfies:

$$c'_k(v) = \text{value}$$

Definition 30 (Triggering getData) Given a system architecture $\text{sys} = (\text{Classes}, \text{Cases})$, a global state $S_{\text{sys}} = (\mathcal{C}, \mathbf{S})$, a *CaseTask* instance $\tau = (\text{case}, t) \in T_{\text{sys}}$ where $\text{case} = (PCC, id) \in \text{Cases}$ and $t \in T_{PCC}$, and a target variable-case pair $(\text{case}_k, v) \in \text{Interface}^*(\tau, S_{\text{sys}})$:

1. **Triggerability:**

The event $e = \text{getData}(\text{case}, t, (\text{case}_k, v)) \in \mathcal{E}_{\text{data}}$ is triggerable under architecture sys and state S_{sys} , expressed by $\text{triggerable}(\text{sys}, S_{\text{sys}}, e) = \text{true}$, if and only if all of the following conditions hold:

$$\begin{aligned} s_{\text{assigned}}(t) &\neq \text{null} && (\text{Assigned Task}) \\ (\text{case}_k, v) &\in \text{Interface}^*(\tau, S_{\text{sys}}) && (\text{Interface Scope}) \end{aligned}$$

where $s = \mathbf{S}(\text{case})$.

2. **State Transition:** When e is triggered, it leaves the system architecture and state completely unchanged, yielding $(\text{sys}', S'_{\text{sys}}) = (\text{sys}, S_{\text{sys}})$, denoted as:

$$\text{trigger}(\text{sys}, S_{\text{sys}}, e) = (\text{sys}, S_{\text{sys}})$$

Execution of the getData event returns the value $\text{val}_{S_{\text{sys}}, \text{case}_k}(v)$.

While the setData event modifies the valuation of a state variable across instance or class scopes, the setAllowedData event manages local form access constraints at the task level. Executing $\text{setAllowedData}((\text{case}, t), w, D) \in \mathcal{E}_{\text{data_allow}}$ dynamically reconfigures the domain of permissible values D for a specific variable w within the local interface $\text{Interface}_{PCC}(t)$ of an assigned task instance t .

Unlike cross-case data mutations, this event operates strictly within the active source context ($target_case(e) = source_case(e) = case$). To maintain structural and transactional consistency, two key invariants are enforced:

1. **Ownership and Permission Scope:** The user authorization of event requires active ownership of task t by user u ($s_{assigned}(t) = u$), while triggerability only checks editability rights over variable w ($s_{editable}^*(w) = 1$).
2. **Active Valuation Consistency:** The transition is triggerable if and only if the current active valuation of the variable $val_{case}(w)$ (whether stored as an instance-level variable in $CaseVars_{PCC}$ or a class-level variable in $ClassVars_{PCC}$) remains valid within the newly specified allowed domain D .

A critical design feature of our model is the strict decoupling of user authorization from system triggerability for permission mutation events. While manual execution of $setAllowedData((case, t), w, D) \in \mathcal{E}_{data_allow}$ by a user u strictly requires active task assignment ($s_{assigned}(t) = u$) within the authorization layer, the system-level predicate $triggerable(sys, S_{sys}, e)$ purposefully omits any assignment check.

This asymmetry allows the execution engine to dynamically reconfigure local interface permissions via automated event cascades (e.g., in response to an assignment event or pre-assign hook) even before an assigned owner is attached to the task ($s_{assigned}(t) = \perp$).

If this constraint is satisfied, the event updates exclusively the local interface state component $s_{allowed}(t)(w)$ of task t in Definition 8, leaving all other task states, class blueprints, and case valuations untouched. Any attempt to set a domain D that excludes the active value $val_{case}(w)$ causes $triggerable$ to fail, yielding $\perp$ and forcing an immediate transactional rollback, as will be explained later.

Definition 31 (Triggering $setAllowedData$) Given a system architecture $sys = (Classes, Cases)$, a global state $S_{sys} = (\mathcal{C}, \mathbf{S})$, a user $u \in Users$, a *CaseTask* instance $\tau = (case, t) \in T_{sys}$ with $case = (PCC, id)$, a variable $w \in Interface_{PCC}(t)$, and a domain subset $D \subseteq Val_{S_{sys}}(Typing_{PCC}(w))$:

1. *Triggerability:*

The event $e = setAllowedData((case, t), w, D) \in \mathcal{E}_{data_allow}$ is triggerable under architecture sys and state S_{sys} , expressed by $triggerable(sys, S_{sys}, e) = true$, if and only if all of the following conditions hold:

$$w \in Interface_{PCC}(t) \quad (\text{Task Interface Scope})$$

$$D \neq s_{allowed}(t)(w) \quad (\text{State Change Necessity})$$

$$val_{case}(w) \in D \quad (\text{Active Valuation Consistency})$$

where $s = \mathbf{S}(case)$.

2. State Transition: When e is triggered, it yields an unchanged architecture $sys' = sys$ and an updated global state $S'_{sys} = (\mathcal{C}, \mathbf{S}')$, denoted as $(sys', S'_{sys}) = \text{trigger}(sys, S_{sys}, e)$, where class states remain unchanged ($\mathcal{C}' = \mathcal{C}$) and $\mathbf{S}'$ differs from $\mathbf{S}$ only at $\text{case} = (PCC, id)$. The updated case state $s' = \mathbf{S}'(\text{case})$ satisfies:

$$s'(t) = (s_{assigned}(t), s_{editable}(t), s_{required}(t), s'_{allowed}(t), s_{consumed}(t))$$

where:

$$s'_{allowed}(t)(x) = \begin{cases} D & \text{if } x = w \\ s_{allowed}(t)(x) & \text{otherwise} \end{cases}$$

Remark 32 (Effective Domain Propagation in Extended Interfaces) Note that while the state transition of `setAllowedData` explicitly mutates only the local task state component $s_{allowed}(t)(w)$ of the source task t , it implicitly affects the effective domain $s^*_{allowed}(w)$ across all active tasks that share variable w within the case context case .

Because effective allowed domains are dynamically derived as the intersection of local restrictions over the extended interface, modifying a single task's local domain D immediately restricts or shifts the global effective permission for w . The physical state update remains localized to $s(t)$, whereas the propagation mechanics across the extended interface are evaluated reactively during state queries. A complete formal treatment of dynamic extended domain synchronization is beyond the scope of this paper.

3.4 Interactions between Cases via Transactions of Action Execution and Event Cascades

Definition 33 introduces the operational logic and execution hooks of the MCMC-PS framework, enabling events within a specific case context to trigger deterministic cascades of operations across the entire system. This mechanism forms the structural foundation for inter-case communication, dynamic cross-case interaction and automation, and complex business logic linking independent cases:

- **Sequence of System Events (1):** An *action* is defined as a finite, potentially empty sequence of system-level events ($\mathcal{E}_{sys}^*$). A single operation initiated within one case (such as completing a task or updating a variable) can thus initiate a deterministic cascade of events that propagate into other active case instances—such as mutating cross-case data, instantiating child cases, or assigning *CaseTasks* in related cases.
- **Cross-Case Pre-actions and Post-actions (2):** Every system event $e \in \mathcal{E}_{sys}$ is encapsulated by two operational logic hooks capable of spanning case boundaries:

- **Pre-action (*pre*):** A state-dependent mapping $pre : \mathcal{S}_{sys} \times \mathcal{E}_{sys} \rightarrow \mathcal{E}_{sys}^*$ that computes a sequence of system events to be evaluated and executed *prior* to the state transition of the primary event. This is typically used to validate conditions across related cases or prepare the global context before executing an event.
- **Post-action (*post*):** A state-dependent mapping $post : \mathcal{S}_{sys} \times \mathcal{E}_{sys} \rightarrow \mathcal{E}_{sys}^*$ that determines the sequence of system events to be executed *after* the successful state transition of the primary event. This hook manages cross-case process chaining, automated state propagation, and side effects in external cases.

Definition 33 (Actions) *Given a system architecture $sys = (Classes, Cases)$, the domain of all global system states $\mathcal{S}_{sys}$, and the unified set of all system-level events $\mathcal{E}_{sys} = \mathcal{E}_{task} \cup \mathcal{E}_{case} \cup \mathcal{E}_{data}$, we define actions and state-dependent event hooks as follows:*

1. **Action Domain:** *An action is a finite (and possibly empty) sequence of system-level events, denoted as:*

$$action \in \mathcal{E}_{sys}^*$$

2. **Pre-action and Post-action Hooks:** *Each event $e \in \mathcal{E}_{sys}$ is associated with a pre-action mapping pre and a post-action mapping $post$:*

- (a) $pre : \mathcal{S}_{sys} \times \mathcal{E}_{sys} \rightarrow \mathcal{E}_{sys}^*$ *is a function assigning a pre-action sequence to a given global state and event.*
- (b) $post : \mathcal{S}_{sys} \times \mathcal{E}_{sys} \rightarrow \mathcal{E}_{sys}^*$ *is a function assigning a post-action sequence to a given global state and event.*

Definition 34 describes the recursive transactional execution engine of the MCMC-PS framework. It formalizes how a user-initiated interaction triggers a cascading chain of automated events across cases, ensuring transactional integrity through a fail-fast mechanism:

- **User-Initiated Transaction Entry:** The entry point *execute_transaction* strictly enforces user authorization via $authorized(sys, S_{sys}, e, u)$ and structural readiness via $triggerable(sys, S_{sys}, e)$ for the primary event $e \in \mathcal{E}_{sys}$. If authorized and triggerable, execution proceeds; if any sub-event in the resulting execution tree fails, the inner engine returns $\perp$, causing the entry function to roll back and return the unaltered initial state S_{sys} .
- **Recursive Cascading Logic:** Once initialized, execution is governed by a set of mutually recursive functions (*execute_event_with_actions*, *execute*, *execute_sequence*).

When an event e is triggered, the system first executes its pre-actions ($pre(e)$), then the core event transition itself ($trigger(sys, S_{sys}, e)$), and finally its post-actions ($post(e)$).

- **Transactional Stability ($\perp$):** The symbol $\perp \notin \mathcal{S}_{sys}$ represents an invalid or failed state. If any event in the entire execution tree—whether a pre-action, the main event, or a deeply nested post-action—cannot be triggered ($triggerable$ evaluates to *False*), the entire execution fails and evaluates to $\perp$.
- **Deep Cascading and Non-Termination:** Because *execute_sequence* transitively calls *execute_event_with_actions* for each sub-event, event chains can span across related case boundaries. Ensuring that these cascades terminate and do not result in infinite loops remains the responsibility of the process designer.

Definition 34 (Transaction Execution Engine) *Let sys be an MCMC-PS architecture, $\mathcal{S}_{sys}$ the set of global system states, $\mathcal{E}_{sys}$ the set of system events, and $\perp \notin \mathcal{S}_{sys}$ representing the failure state. Using the helper functions defined below, the transactional execution of an event $e \in \mathcal{E}_{sys}$ initiated by user $u \in Users$ is formalised as:*

```

1 execute_transaction( $sys, S_{sys} \in \mathcal{S}_{sys}, u \in Users, e \in \mathcal{E}_{sys}$ ) {
2   if  $authorized(sys, S_{sys}, e, u) \wedge triggerable(sys, S_{sys}, e)$  {
3      $S'_{sys} \leftarrow \text{execute\_event\_with\_actions}(sys, S_{sys}, e)$ 
4     if  $S'_{sys} \neq \perp$  return  $S'_{sys}$ 
5   }
6   return  $S_{sys}$ 
7   // Rollback to initial state on authorization, triggerability, or action failure
8 }
9
10 execute_event_with_actions( $sys, S_{sys} \in \mathcal{S}_{sys} \cup \{\perp\}, e \in \mathcal{E}_{sys}$ ) {
11   if  $triggerable(sys, S_{sys}, e)$  {
12      $S_{sys} \leftarrow \text{execute\_sequence}(sys, S_{sys}, pre(e))$ 
13      $S_{sys} \leftarrow \text{execute}(sys, S_{sys}, e)$ 
14      $S_{sys} \leftarrow \text{execute\_sequence}(sys, S_{sys}, post(e))$ 
15     return  $S_{sys}$ 
16   }
17   return  $\perp$ 
18 }
19
20 execute( $sys, S_{sys} \in \mathcal{S}_{sys} \cup \{\perp\}, e \in \mathcal{E}_{sys}$ ) {
21   if  $triggerable(sys, S_{sys}, e)$ 
22     return  $trigger(sys, S_{sys}, e)$ 
23   else
24     return  $\perp$ 

```

```

25 }
26
27 execute_sequence( $sys, S_{sys} \in \mathcal{S}_{sys} \cup \{\perp\}, \sigma \in \mathcal{E}_{sys}^*$ ) {
28   for ( $e$  in  $\sigma$ ) { // iteratively execute each event in sequence  $\sigma$ 
29      $S_{sys} \leftarrow$  execute_event_with_actions( $sys, S_{sys}, e$ )
30   }
31   return  $S_{sys}$ 
32 }

```

Finally, for any state $S_{sys} \in \mathcal{S}_{sys}$, user $u \in Users$, and event $e \in \mathcal{E}_{sys}$:

event e is transactionally executable by user u in state S_{sys}

$\iff$

$authorized(sys, S_{sys}, e, u) \wedge$

$triggerable(sys, S_{sys}, e) \wedge$

$\mathbf{execute_event_with_actions}(sys, S_{sys}, e) \neq \perp$

Chapter 4

Inter-Process Orchestration & Batch Task Operations

While basic process patterns rely on static control flows and isolated process instances, real-world enterprise management often demands dynamic interactions across multiple autonomous entities. This section briefly outlines a typical application implemented by Petriflow PCCs, namely the master's thesis management scenario that demonstrates orchestration across three separate process-centric class definitions: **Student**, **Thesis**, and **Committee**.

The interaction relies on multi-instance state synchronization and batch task manipulation rather than centralized execution. The system three distinct Petriflow classes:

- **Student:** Governs the student's workflow from selecting a topic to submitting the thesis and checking the final grade.
- **Thesis:** Tracks the lifecycle of the thesis document created by a supervisor (**Topic** task), including topic reservation, document submission, and grading.
- **Committee:** Acts as an aggregation process that combines multiple submitted theses into a single examination session.

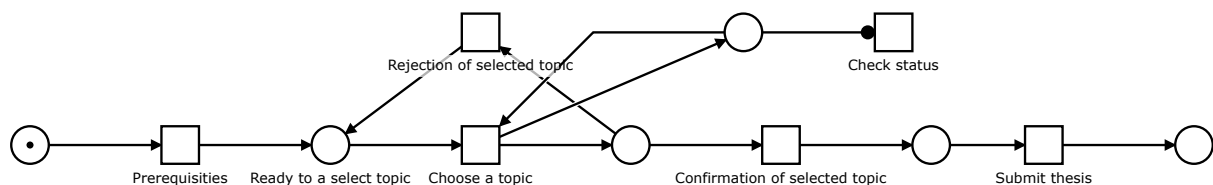


Figure 4.1: Student class.

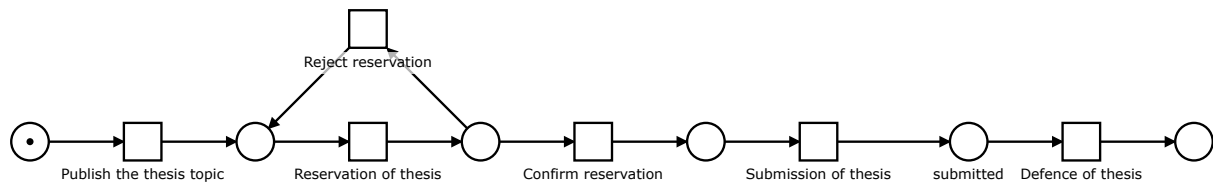


Figure 4.2: Thesis class.

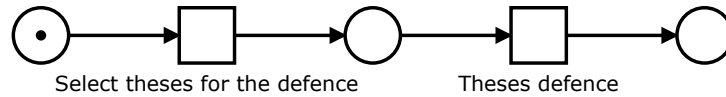


Figure 4.3: Committee class

4.1 Batch Task Reference & Interface Aggregation

The most complex pattern in this application occurs during the defense phase managed by the **Committee** class. In the **Select theses for the defence** task, the committee chair selects candidate theses scheduled for defense using a `multichoice_map` field named `submitted_theses` (see Listing 1).

```

1 def theses = findCases {
2   it.processIdentifier.eq("Thesis")
3   .and(it.activePlaces.get("submitted").eq(1))
4 }.collectEntries {
5   [(it.stringId): "Thesis name: " + it.getFieldValue("thesis_name") +
6     " Student: " + it.getFieldValue("student_name") + " " +
7     it.getFieldValue("student_surname")]
8 }
9 change submitted_theses options { theses }

```

Listing 1: Dynamic collection of all submitted theses ready for defence (cases with process identifier `Thesis` and token in place `submitted`) and changing of allowed options of the field `submitted_theses`.

On pre-action of *finishTask* event of the task **Select theses for the defence** of the case of the **Committee** class, the `submitted_theses.value` (which consists of IDs of selected theses for the defence) is used to find tasks called **Defence of thesis** (with task id `defence`) of the cases of the class **Thesis**. These **Defence of thesis** tasks are collected and saved in the variable `theses_defence` of the **Committee** case (see Listing 2).

In the subsequent **Theses defence** task of the **Committee** case, the user interfaces of all referenced **Defence of thesis** tasks (with id `defence`) of **Thesis** cases are embedded directly into the extended interface *Interface** of the **Theses defence** task through the taskRef variable `theses_defence` as subforms. This allows the committee members to evaluate and grade all selected theses within a single interface of **Theses defence** task of

```

1 change theses_defence value {
2   findTasks {
3     it.caseId.in(submitted_theses.value)
4     .and(it.transitionId.eq("defence"))
5   }?.collect { it.stringId }
6 }

```

Listing 2: Dynamic retrieval of embedded defence tasks (tasks with title **Defence of thesis** and id **defence**) on pre-action of *finishTask* event of the task **Select theses for the defence**.

the **Committee** case.

Furthermore, on pre-action of *assignTask* event of **Theses defence** task of the **Committee** case, all **Defence of thesis** tasks of selected **Thesis** cases are assigned too by their *assignTask* event (see Listing 3). This is done transactionally, simulating synchronous occurrence of all these *assignTask* events.

```

1 theses_defence.value.each { id ->
2   assignTask(findTask(id))
3 }

```

Listing 3: Transactional task assignment across process instances during the *assignTask* event of **Theses defence** task of the **Committee** case.

Analogous transactions are executed on the *finish* and *cancel* events of **Theses defence** task of the **Committee** case using *finishTask* and *cancelTask* respectively. This pattern highlights the capability of the Petriflow architecture to model Multi-Class Multi-Case enterprise applications where a case of aggregating process-centric class (**Committee**) securely and transactionally controls multiple child cases of processes-centric class **Thesis** that synchronize their states with cases of class **Student**.

4.2 Query Mechanics and Abstract Addressability

In implementation environments (such as Petriflow XML), locating target cases or tasks relies on imperative query actions (e.g., **findCase** or **findTask**) that dynamically evaluate predicates over the global system state S_{sys} . In theoretical formalization presented in this paper, we deliberately abstract away the explicit algebraic syntax of such query expressions.

Because every active case instance (PCO, id) and task t possesses a unique global address within MCMC-PS, any query predicate fundamentally evaluates to a deterministic subset of global instances or task identities. Consequently, actions that build event sequences $\sigma \in \mathcal{E}_{sys}^*$ operate directly on case of those events. This separation ensures that

the transactional execution semantics remain strictly clean, modular, and decoupled from the underlying query evaluation runtime.

Chapter 5

Discussion: Architectural Foundations & Comparative Analysis

The architectural paradigm of Multi-Case Petriflow Systems (MCPS) fundamentally diverges from classical Petri net composition frameworks, such as Desel’s interface-based net composition [Des25; DK01; Des26] and Reisig’s *Heraklit* [FR24; FR20].

5.1 Composition vs. Autonomous Interaction

In traditional compositional semantics ([FR24; FR20], Desel), composition relies on merging interface transitions or places across net boundaries. Once two modules are composed, the interface elements often become encapsulated within the internal structure of the newly formed composite module. This structural fusion hides original composition boundaries, making decomposition impossible without maintaining an explicit hierarchical tree of nested modules.

In Petriflow, we deliberately reject structural net composition in favor of **autonomous instance interaction**, inspired by Object-Oriented Programming (OOP) and relational data models. As demonstrated by the *Thesis defence* example, the interacting cases are not merged into a single composite Petri net. Instead, each case remains a fully autonomous, encapsulated instance. The **Committee** case acts as an orchestrating mediator rather than a structural container.

5.2 Communication vs. Transactional Interaction

We establish a strict semantic distinction between messaging, communication, and interaction:

- **Communication / Messaging:** Unidirectional message passing or asynchronous request dispatching, which lacks native transactional boundaries.

- **Transactional Interaction:** Multi-case interactions in Petriflow are fundamentally *atomic and transactional*. Interacting instances retain their autonomy, but their joint state transitions are evaluated under strict all-or-nothing transactional guarantees.

5.3 Interleaved Event Cascading and Synchronous Step Simulation

Instead of requiring global multi-net synchronous steps—which demand complex lock allocation across all participating events—Petriflow achieves multi-case synchronization via ****interleaved event sequences** (σ_{cascade}) evaluated under optimistic locking^{**}:

1. **Equivalence to Synchronous Steps:** If two target task events in distinct case instances are structurally independent, their execution order is commutative. Thus, executing them sequentially within an atomic event transaction (`execute_sequence`) semantically simulates a synchronous multi-case step.
2. **Optimistic Locking vs. Global Locking:** Traditional synchronous multi-net steps require computing independence relations upfront and acquiring global multi-entity locks across all involved net events. Petriflow’s optimistic locking approach eliminates this overhead: actions attempt to execute sequential cascades dynamically, rolling back to $\perp$ if a state invariant or dynamic validation fails.

5.4 Interval Transition Semantics and Call-by-Name / Call-by-Value Data Forms

Petriflow employs **interval transition semantics** (or a priori semantics in the sense of [AJ15]), where a task execution spans a non-instantaneous interval defined by explicit lifecycle events: *assignTask* (start/test), data interactions (*setData*), and *finishTask* (completion/consume-and-produce).

- **Limitation on Intra-Class Synchronous Steps:** Because a task requires testing and consuming tokens across an interval rather than an instantaneous firing step, synchronous steps between transitions *within a single class* cannot be directly executed in a single atomic tick.
- **User Interface Data Binding:** User interface forms bound to tasks utilize a *call-by-name* mechanism for variables of the case. We are working on a possible *call-by-value* mechanism for variables in task interfaces, so-called **dataRefs**. Under this mechanism, upon task assignment (*assignTask*), values are transferred to

copies of variables called **dataRefs**; these **dataRefs** are updated via **setData**, and upon completion (*finishTask*), the values of these **dataRefs** are copied back to the reference variables.

5.5 Expressiveness Trade-offs

The primary trade-off of Petriflow’s strict object encapsulation is that an instance (e.g., Case A) cannot natively connect arcs directly to the places of another instance (Case B). To inspect or reflect the state of a place from Case A within Case B without violating encapsulation, MCMC-PS utilizes a state-synchronization pattern via dynamic variable arcs. Specifically, a target place is duplicated in Case B and coupled with dedicated **add** and **remove** transitions connected via variable-weight arcs bound to data variables. Whenever a transition in Case A modifies the token count of its local place, its associated post-action updates the data variable to match the added or removed tokens and synchronously triggers the corresponding **add** or **remove** transition in Case B, perfectly replicating the token count in the mirrored place. In software engineering and low-code practice, this explicit event-driven synchronization preserves class modularity, interface boundaries, and microservice-friendly runtime independence.

Auto-Concurrency vs. Explicit Task Lifecycle

A critical architectural distinction between classical Petri net theory and the practical execution semantics of MCMC-PS lies in the handling of auto-concurrency. In standard Petri nets, multiple token combinations in input places can theoretically yield simultaneous parallel activations of a single transition t . In a case-driven process engine, however, unconstrained auto-concurrency degrades interface determinism and fragments local task states—such as $s_{assigned}$, $s_{editable}$, and cross-case variable bindings—across concurrent instances of the same task within a single case.

By prohibiting auto-concurrency, MCMC-PS enforces a strict bijection between a case-task pair $((PCC, id), t)$ and its global task representation $\tau \in T_{sys}$. Rather than dynamically creating and destroying task objects upon token arrivals, tasks continuously exist within the local state $s(t)$ and transition through an explicit lifecycle (**disabled** $\rightarrow$ **enabled** $\rightarrow$ **assigned** $\rightarrow$ **finished**). This persistent model ensures that even while **disabled**, tasks preserve their local data state, input token accumulations ($s_{consumed}$), and cross-case reference configurations, guaranteeing deterministic state resolution during dynamic event execution.

Chapter 6

Conclusion

In this paper, we extended the formal semantics of Petriflow to Multi-Case Petriflow Systems (MCPS), bridging the gap between theoretical High-Level Petri nets and low-code software engineering.

This paper establishes the formal reference semantics of the Petriflow language, which is actively used in industrial practice. While concrete illustrative examples and domain-specific case studies are provided in existing works such as [Juh+26] as well as follow-up publications, this work serves as the self-contained foundational framework for multi-class multi-case Petriflow systems.

Unlike classical compositional frameworks (such as Desel’s net composition [Des25; DK01; Des26] or Reisig’s *Heraklit* [FR24; FR20]) that merge nets through shared elements and hide structural boundaries, Petriflow rejects structural net fusion. Instead, it enforces strict object-oriented encapsulation where process instances remain fully autonomous. By distinguishing non-transactional communication from atomic interaction, MCPS introduces a transactional execution engine. Using interleaved event cascades evaluated under optimistic locking, Petriflow simulates multi-case synchronous steps without requiring heavy upfront global locking or complex independence calculations.

Furthermore, by combining interval transition semantics (a priori testing, consumption, and production) with call-by-name and call-by-value dynamic interface binding, Petriflow unifies process execution, data modeling, and user interface rendering within a single framework. While strict encapsulation intentionally prevents direct cross-case token manipulation, this trade-off ensures high modularity, scalability, and reference integrity. Ultimately, MCPS establishes a mathematically sound yet highly practical foundation for building modern, distributed, and executable process-centric applications.

Future Work

Building upon the formalization of Multi-Class Multi-Case Petriflow Systems (MCMC-PS) presented in this work, two primary directions for future research emerge. First, we intend

to construct a rigorous, full-fledged formal proof demonstrating the strict one-to-one semantic equivalence between the underlying Petri net execution semantics—including event enablings, guard expressions, and time-dependent fireability—and the state transition relations defined for system-level events. Second, from an architectural and theoretical perspective, we aim to explore a fully homogeneous system model where the explicit, external set of users (*Users*) is completely eliminated. In this unified paradigm, users are naturally specified as standard Petriflow classes (*PCC*), and all user interactions, role assignments, and event triggers are re-formulated purely as dynamic inter-case operations. This abstraction promises to further simplify the formal execution model while maintaining complete operational expressiveness.

Bibliography

- [AB20] Wil M. P. van der Aalst and Alessandro Berti. “Discovering Object-Centric Petri Nets”. In: *CoRR* abs/2010.02047 (2020). DOI: 10.48550/arXiv.2010.02047. arXiv: 2010.02047.
- [Aal+09] Wil M. P. van der Aalst et al. “Soundness of Workflow Nets with Reset Arcs”. In: *Transactions on Petri Nets and Other Models of Concurrency III*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 50–70. ISBN: 978-3-642-04856-2. DOI: 10.1007/978-3-642-04856-2_3.
- [AJ15] Mohammed Alqarni and Ryszard Janicki. “On Interval Process Semantics of Petri Nets with Inhibitor Arcs”. In: *Application and Theory of Petri Nets and Concurrency*. Springer International Publishing, 2015, pp. 77–97. ISBN: 978-3-319-19488-2.
- [Cab+09] Lawrence Cabac et al. “Renew: The Reference Net Workshop”. In: *Transactions on Petri Nets and Other Models of Concurrency II* (2009), pp. 1–25. URL: <https://ceur-ws.org/Vol-1372/paper18.pdf>.
- [Des25] Jörg Desel. “Composing Chameleons.” In: *PNSE/ATAED/PeNGE@ PETRI NETS*. 2025, pp. 217–223.
- [Des26] Jörg Desel. “Interaction of Petri Net Process Models”. In: *Mining a Scientist’s Process: Essays Dedicated to Wil van der Aalst on the Occasion of His 60th Birthday*. Springer, 2026, pp. 284–297.
- [DK01] Jörg Desel and Ekkart Kindler. “Petri nets and components—Extending the DAWN approach”. In: *Workshop on Modelling of Objects, Components, and Agents*. 2001, pp. 21–35.
- [FR20] Peter Fettke and Wolfgang Reisig. “Modelling service-oriented systems and cloud services with Heraklit”. In: *European Conference on Service-Oriented and Cloud Computing*. Springer. 2020, pp. 77–89.
- [FR24] Peter Fettke and Wolfgang Reisig. *Understanding the digital world: modeling with Heraklit*. Springer Nature, 2024.

- [GA05] Christian W Günther and Wil MP van der Aalst. “Modeling the case handling principles with colored Petri nets”. In: *Proceedings of the Sixth Workshop on the Practical Use of Coloured Petri Nets and CPN Tools (CPN 2005)*. Vol. 576. Citeseer. 2005, pp. 211–230.
- [JK09] Kurt Jensen and Lars M. Kristensen. *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer Science & Business Media, 2009. DOI: 10.1007/b95112.
- [JLM07] Gabriel Juhás, Robert Lorenz, and Sebastian Mauser. “Complete Process Semantics for Inhibitor Nets”. In: *Petri Nets and Other Models of Concurrency – ICATPN 2007*. Ed. by Jetty Kleijn and Alex Yakovlev. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 184–203. ISBN: 978-3-540-73094-1. DOI: https://doi.org/10.1007/978-3-540-73094-1_13.
- [Juh+25] Gabriel Juhás et al. “Petriflow: A Low-Code Language for Object Centric Processes”. In: *AWPN 2025 workshop proceedings*. Ed. by Robert Lorenz, Robin Bergenthum, and Patrizia Schalk. Bonn: Gesellschaft für Informatik e.V., 2025, pp. 3–22. DOI: 10.18420/AWPN2025-01.
- [Juh+26] Gabriel Juhás et al. “Netgrif Platform: A Tool for Executable Models of Object-Centric Processes in Petriflow Language”. In: *Application and Theory of Petri Nets and Concurrency*. Springer Nature Switzerland, 2026, pp. 371–382. ISBN: 978-3-032-27879-1. DOI: 10.1007/978-3-032-27879-1_19.
- [KWM04] Olaf Kummer, Frank Wienberg, and Daniel Moldt. “Renew: The Reference Net Workshop”. In: *Tool Presentations, 25th International Conference on Applications and Theory of Petri Nets*. 2004.
- [Val78] Rüdiger Valk. “Self-modifying nets, a natural extension of Petri nets”. In: *Automata, Languages and Programming*. Springer Berlin Heidelberg, 1978, pp. 464–476. ISBN: 978-3-540-35807-7. DOI: 10.1007/3-540-08860-1_35.
- [Vog02] Walter Vogler. “Partial order semantics and read arcs”. In: *Theoretical Computer Science* 286.1 (2002). Mathematical Foundations of Computer Science, pp. 33–63. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/S0304-3975\(01\)00234-1](https://doi.org/10.1016/S0304-3975(01)00234-1).

Appendix A

Supplementary Material

For newest version of this report and other supplementary material including XML scheme of the syntax of the Petriflow OCP see petriflow.org.